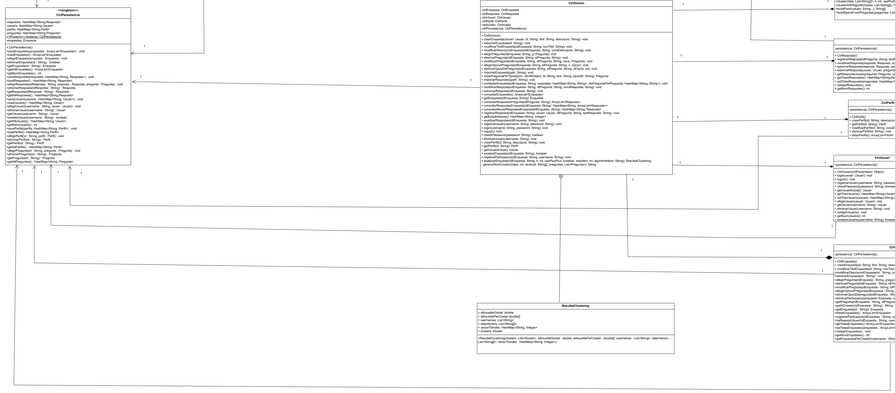

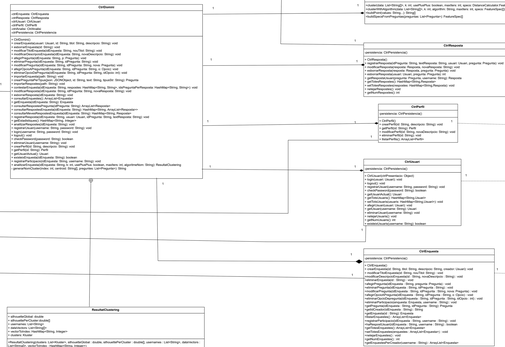

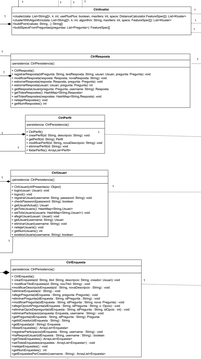

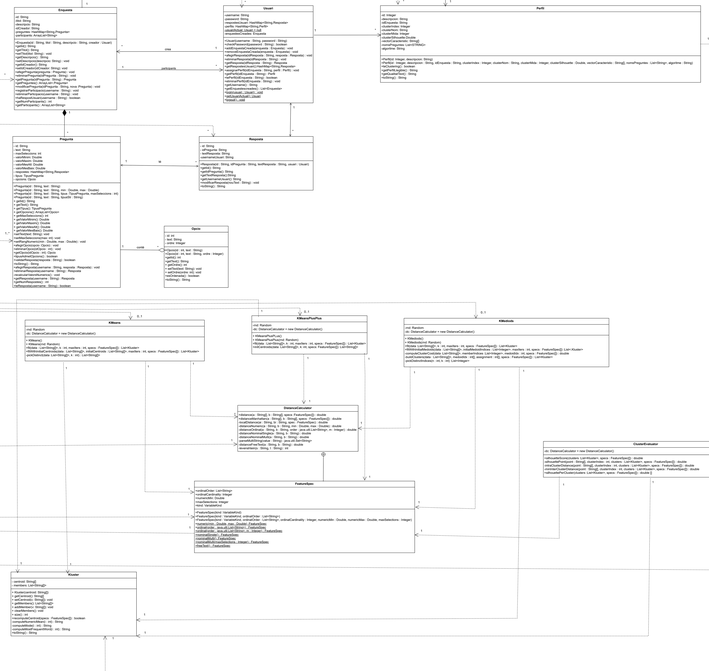

Persistència

Controladors

Classes

crea
participants
té
conté
1
*
0..1
1..*

<<singleton>>
CtrlPersistencia

-respostes: HashMap<String,Resposta>
-usuaris: HashMap<String,Usuari>
-perfils: HashMap<String,Perfil>
-preguntes: HashMap<String,Pregunta>
<<Property>>-Instance: CtrlPersistencia
-enquestes: Enquesta

+ CtrlPersistencia()
+saveEnquestes(enquestes : ArrayList<Enquesta>) : void
+loadEnquestes() : ArrayList<Enquesta>
+afegirEnquesta(enquesta : Enquesta) : void
+eliminarEnquesta(id : String) : boolean
+getEnquesta(id : String) : Enquesta
+getAllEnquestes() : ArrayList<Enquesta>
+getNumEnquestes() : int
+saveRespostes(respostes :HashMap<String, Resposta>) : void
+loadRespostes() : HashMap<String, Resposta>
+afegirResposta(idResposta : String, resposta : Resposta, pregunta : Pregunta) : void
+eliminarResposta(idResposta : String) : Resposta
+getResposta(idResposta : String) : Resposta
+getAllRespostes() : HashMap<String,Resposta>
+saveUsuaris(usuaris : HashMap<String, Usuari>) : void
+loadUsuaris() : HashMap<String, Usuari>
+afegirUsuari(username : String, usuari : Usuari) : void
+eliminarUsuari(username : String) : Usuari
+getUsuari(username : String) : Usuari
+existeixUsuari(username : String) : boolean
+getAllUsuaris() : HashMap<String, Usuari>
+getNumUsuaris() : int
+savePerfils(perfils :HashMap<String, Perfil>) : void
+loadPerfils() :HashMap<String,Perfil>
+afegirPerfil(id : String, perfil : Perfil) : void
+eliminarPerfil(id : String) : Perfil
+getPerfil(id : String) : Perfil
+getAllPerfils() : HashMap<String, Perfil>
+afegirPregunta(id : String, pregunta : Pregunta) : void
+eliminarPregunta(id : String) : Pregunta
+getPregunta(id : String) : Pregunta
+getAllPreguntes() : HashMap<String, Pregunta>

CtrlDomini

-ctrlEnquesta: CtrlEnquesta
-ctrlResposta: CtrlResposta
-ctrlUsuari: CtrlUsuari
-ctrlPerfil: CtrlPerfil
-ctrlAnalisi: CtrlAnalisi
-ctrlPersistencia: CtrlPersistencia

+ CtrlDomini()
+ crearEnquesta(usuari: Usuari, id: String, titol: String, descripcio: String): void
+ esborrarEnquesta(id: String): void
+ modificarTitolEnquesta(idEnquesta: String, nouTitol: String): void
+ modificarDescripcioEnquesta(idEnquesta: String, novaDescripcio: String): void
+ afegirPregunta(idEnquesta: String, p: Pregunta): void
+ eliminarPregunta(idEnquesta: String, idPregunta: String): void
+ modificarPregunta(idEnquesta: String, idPregunta: String, nova: Pregunta): void
+ afegirOpcioAPregunta(idEnquesta: String, idPregunta: String, o: Opcio): void
+ eliminarOpcioDePregunta(idEnquesta: String, idPregunta: String, idOpcio: int): void
+ importarEnquesta(path: String): void
+ crearPreguntaPerTipus(json: JSONObject, id: String, text: String, tipusStr: String): Pregunta
+ importarRespostes(path: String): void
+ contestarEnquesta(idEnquesta: String, respostes: HashMap<String, String>, idsPreguntaPerResposta: HashMap<String, String>): void
+ modificarResposta(idEnquesta: String, idPregunta: String, novaResposta: String): void
+ esborrarResposta(idEnquesta: String): void
+ consultarEnquestes(): ArrayList<Enquesta>
+ getEnquesta(idEnquesta: String): Enquesta
+ consultarRespostesPregunta(idPregunta: String): ArrayList<Resposta>
+ consultarRespostesEnquesta(idEnquesta: String): HashMap<String, ArrayList<Resposta>>
+ consultarMevesRespostesEnquesta(idEnquesta: String): HashMap<String, Resposta>
+ registrarResposta(idEnquesta: String, usuari: Usuari, idPregunta: String, textResposta: String): void
+ getEstadistiques(): HashMap<String, Integer>
+ analitzarRespostes(idEnquesta: String): void
+ registrarUsuari(username: String, password: String): void
+ login(username: String, password: String): void
+ logout(): void
+ checkPassword(password: String): boolean
+ eliminarUsuari(username: String): void
+ crearPerfil(id: String, descripcio: String): void
+ getPerfil(id: String): Perfil
+ getUsuariActual(): Usuari
+ existeixEnquesta(idEnquesta: String): boolean
+ registrarParticipacio(idEnquesta: String, username: String): void
+ analitzarEnquesta(idEnquesta: String, k: int, usePlusPlus: boolean, maxIters: int, algoritmeNom: String): ResultatClustering
- generarNomCluster(index: int, centroid: String[], preguntes: List<Pregunta>): String

ResultatClustering

+ silhouetteGlobal: double
+ silhouettePerCluster:double[]
+ usernames: List<String>
+ dataVectors: List<String[]>
+ vectorToIndex: HashMap<String, Integer>
+ clusters: Kluster

+ResultatClustering(clusters :List<Kluster>, silhouetteGlobal : double, silhouettePerCluster : double[], usernames : List<String>, dataVectors : List<String[]>, vectorToIndex : HashMap<String, Integer>)

CtrlAnalisi

+cluster(data: List<String[]>, k: int, usePlusPlus: boolean, maxIters: int, specs: DistanceCalculator.FeatureSpec[]): List<Kluster>
+clusterWithAlgorithm(data: List<String[]>, k: int, algorithm: String, maxIters: int, specs: FeatureSpec[]): List<Kluster>
+buildPoint(values: String...): String[]
+buildSpecsFromPreguntas(preguntas: List<Pregunta>): FeatureSpec[]

CtrlResposta

-persistencia: CtrlPersistencia()

+ CtrlResposta()
+ registrarResposta(idPregunta: String, textResposta: String, usuari: Usuari, pregunta: Pregunta): void
+ modificarResposta(resposta: Resposta, novaResposta: String): void
+ esborrarResposta(resposta: Resposta, pregunta: Pregunta): void
+ esborrarResposta(usuari: Usuari, pregunta: Pregunta): int
+ getRespostaUsuari(pregunta: Pregunta, username: String): Resposta
+ getTotesRespostes(): HashMap<String,Resposta>
+ setTotesRespostes(respostes: HashMap<String,Resposta>): void
+ netejarRespostes(): void
+ getNumRespostes(): int

CtrlPerfil

-persistencia: CtrlPersistencia()

+ CtrlPerfil()
+ crearPerfil(id: String, descripcio: String): void
+ getPerfil(id: String): Perfil
+ modificarPerfil(id: String, novaDescripcio: String): void
+ eliminarPerfil(id: String): void
+ llistarPerfils(): ArrayList<Perfil>

CtrlUsuari

-persistencia: CtrlPersistencia()

+ CtrlUsuari(ctrlPresentacio: Object)
+ login(usuari: Usuari): void
+ logout(): void
+ registrarUsuari(username: String, password: String): void
+ checkPassword(password: String): boolean
+ getUsuariActual(): Usuari
+ getTotsUsuaris(): HashMap<String,Usuari>
+ setTotsUsuaris(usuaris: HashMap<String,Usuari>): void
+ afegirUsuari(usuari: Usuari): void
+ getUsuari(username: String): Usuari
+ eliminarUsuari(username: String): void
+ netejarUsuaris(): void
+ getNumUsuaris(): int
+ existeixUsuaris(username: String): boolean

CtrlEnquesta

-persistencia: CtrlPersistencia()

+ CtrlEnquesta()
+ crearEnquesta(id: String, titol: String, descripcio: String, creador: Usuari): void
+ modificarTitolEnquesta(id: String, nouTitol: String) : void
+modificarDescripcioEnquesta(id : String, novaDescripcio : String) : void
+eliminarEnquesta(id : String) : void
+afegirPregunta(idEnquesta : String, pregunta :Pregunta) : void
+eliminarPregunta(idEnquesta : String, idPregunta : String) : void
+modificarPregunta(idEnquesta : String, idPregunta : String, nova: Pregunta) : void
+afegirOpcioAPregunta(idEnquesta : String, idPregunta : String, o :Opcio) : void
+eliminarOpcioDepregunta(idEnquesta : String, idPregunta : String, idOpcio : int) : void
+eliminarParticipacio(enquesta: Enquesta, username : String) : void
+getPregunta(idEnquesta : String, idPregunta : String): Pregunta
+getIdCreador(idEnquesta : String) : String
+getEnquesta(id : String): Enquesta
+llistarEnquestes() : ArrayList<Enquesta>
+registrarParticipacio(idEnquesta : String, username : String) : void
+haRespostUsuari(idEnquesta : String, username : String) : boolean
+getTotesEnquestes() :ArrayList<Enquesta>
+setTotesEnquestes(enquestes : ArrayList<Enquesta>) : void
+netejarEnquestes() : void
+getNumEnquestes() : int
+getEnquestesPerCreador(username : String) : ArrayList<Enquesta>

Enquesta

-id: String
-titol: String
-descripcio: String
-idCreador: String
-preguntes: HashMap<String,Pregunta>
-participants: ArrayList<String>

+Enquesta(id : String, titol : String, descripcio : String, creador : Usuari)
+getId(): String
+getTitol(): String
+setTitol(titol: String) : void
+getDescripcio() : String
+setDescripcio(descripcio: String) : void
+getIdCreador(): String
+setIdCreador(idCreador: String): void
+afegirPregunta(pregunta : Pregunta) : void
+eliminarPregunta(idPregunta : String) : void
+getPregunta(idPregunta : String) : Pregunta
+getPreguntes() :ArrayList<.Pregunta>
+modificarPregunta(idPregunta : String, nova :Pregunta) : void
+registrarParticipacio(username : String) : void
+eliminarParticipacio(username : String) : void
+haRespostUsuari(username : String) : boolean
+getNumParticipants() : int
+getParticipants() : ArrayList<String>

Usuari

-username: String
-password: String
-respostesUsuari: HashMap<String,Resposta>
-perfils: HashMap<String,Perfil>
-usuariActual: Usuari = null
-enquestesCreades: Enquesta

+Usuari(username : String, password : String)
+checkPassword(password : String) : boolean
+addEnquestaCreada(enquesta : Enquesta) : void
+removeEnquestaCreada(enquesta : Enquesta) : void
+afegirResposta(idResposta : String, resposta : Resposta) : void
+eliminarResposta(idResposta : String) : void
+getResposta(idResposta : String) : Resposta
+getRespostesUsuari():HashMap<String,Resposta>
+assignarPerfil(idEnquesta : String, perfil : Perfil) : void
+getPerfil(idEnquesta : String) : Perfil
+tePerfil(idEnquesta : String) : boolean
+eliminarPerfil(idEnquesta : String) : void
+getUsername() : String
+getEnquestescreades() : List<Enquesta>
+login(usuari : Usuari) : void
+getUsuariActual(): Usuari
+logout() : void

Perfil

-id: Integer
-descripcion: String
-idEnquesta: String
-clusterIndex: Integer
-clusterNom: String
-clusterMida: Integer
-clusterSilhouette:Double
-vectorCaracteristic: String[]
-nomsPreguntes: List<STRING>
-algoritme: String

+Perfil(id: Integer, descripcion: String)
+Perfil(id : Integer, descripcion : String, idEnquesta : String, clusterIndex : Integer, clusterNom : String, clusterMida : Integer, clusterSilhouette : Double, vectorCaracteristic : String[], nomsPreguntes : List<String>, algoritme : String)
+teClustering() : boolean
+getPerfilLlegible() : String
+getQualitatText() : String
+toString() : String

<<enumeration>>
TipusPregunta

NUMERICA
QUALITATIVA_ORDENADA
QUALITATIVA_NO_ORDENADA_SIMPLE
QUALITATIVA_NO_ORDENADA_MULTIPLE
TEXT_LLIURE

<<enumeration>>
VariableKind

NUMERIC
ORDINAL
NOMINAL_SINGLE
NOMINAL_MULTI
FREE_TEXT

Pregunta

- id: String
- text: String
- maxSeleccions: int
- valorMinim: Double
- valorMaxim: Double
- valorMesAlt: Double
- valorMesBaix: Double
- respostes: HashMap<String,Resposta>
- tipus: TipusPregunta
- opcions: Opcio

+Pregunta(id : String, text : String)
+Pregunta(id : String, text : String, min : Double, max : Double)
+Pregunta(id : String, text : String, tipus :TipusPregunta, maxSeleccions : int)
+Pregunta(id : String, text : String, tipusStr : String)
+ getId(): String
+ getText(): String
+ getTipus(): TipusPregunta
+ getOpcions(): ArrayList<Opcio>
+ getMaxSeleccions(): int
+ getValorMinim(): Double
+ getValorMaxim(): Double
+ getValorMesAlt(): Double
+ getValorMesBaix(): Double
+setText(text: String): void
+setMaxSeleccions(max: int) :void
+setRangNumeric(min : Double, max : Double) : void
+afegirOpcio(opcio :Opcio) : void
+eliminarOpcio(idOpcio : int) : void
+getOpcio(idOpcio : int) :Opcio
+tipusAdmetOpcions() : boolean
+validarResposta(resposta : String) : boolean
+toString() : String
+afegirResposta(username : String, resposta : Resposta) : void
+eliminarResposta(username : String) : Resposta
-recalcularValorsNumerics() : void
+getResposta(username : String) : Resposta
+getNumRespostes() : int
+teResposta(username : String) : boolean

Resposta

- id: String
- idPregunta: String
- textResposta: String
-usernameUsuari: String

+Resposta(id : String, idPregunta : String, textResposta : String, usuari : Usuari)
+getId(): String
+getIdPregunta(): String
+getTextResposta():String
+getUsernameUsuari(): String
+modificarResposta(nouText : String) : void
+toString() : String

Opcio

- id: int
- text: String
- ordre: Integer

+Opcio(id : int, text : String)
+Opcio(id : int, text : String, ordre : Integer)
+getId(): int
+getText(): String
+ getOrdre(): int
+ setText(text: String): void
+ setOrdre(ordre: int): void
+esOrdenada() : boolean
+toString() : String

KMeans

-rnd: Random
-dc: DistanceCalculator = new DistanceCalculator()

+ KMeans()
+ KMeans(rnd: Random)
+fit(data : List<String[]>, k : int, maxIters : int, specs : FeatureSpec[]) : List<Kluster>
+fitWithInitialCentroids(data : List<String[]>, initialCentroids : List<String[]>, maxIters : int, specs : FeatureSpec[]) : List<Kluster>
-pickDistinct(data :List<String[]>, k : int) : List<String[]>

KMeansPlusPlus

-rnd: Random
-dc: DistanceCalculator = new DistanceCalculator()

+ KMeansPlusPLus()
+ KMeansPlusPlus(rnd: Random)
+fit(data : List<String[]>, k : int, maxIters : int, specs : FeatureSpec[]) : List<Kluster>
+initCentroids(data: List<String[]>, k: int, specs: FeatureSpec[]): List<String[]>

KMedoids

-rnd: Random
-dc: DistanceCalculator = new DistanceCalculator()

+ KMedoids()
+ KMedoids(rnd: Random)
+fit(data :List<String[]>, k : int, maxIters : int, specs : FeatureSpec[]) : List<Kluster>
+fitWithInitialMedoids(data : List<String[]>, initialMedoidIndices : List<Integer>, maxIters : int, specs : FeatureSpec[]) :List<.Kluster>
-computeClusterCost(data :List<String[]>, memberIndices :List<Integer>, medoidIdx : int, specs : FeatureSpec[]) : double
-buildClusters(data : List<String[]>, medoidIdx : int[], assignment : int[], specs : FeatureSpec[]) : List<Kluster>
-pickDistinctIndices(n : int, k : int) :List<Integer>

DistanceCalculator

+distance(a : String[], b : String[], specs :FeatureSpec[]) : double
+distanceManhattan(a : String[], b : String[], specs : FeatureSpec[]) : double
-localDistance(ai : String, bi : String, spec : FeatureSpec) : double
-distanceNumeric(a : String, b : String, min : Double, max : Double) : double
-distanceOrdinal(a : String, b : String, order : java.util.List<String>, m : Integer) : double
-distanceNominalSingle(a : String, b : String) : double
-distanceNominalMulti(a : String, b : String) : double
-parseMultiString(value : String) : java.util.Set<String>
-distanceFreeText(a : String, b : String) : double
-levenshtein(s : String, t : String) : int

ClusterEvaluator

-dc: DistanceCalculator = new DistanceCalculator()

+silhouetteScore(clusters :List<Kluster>, specs : FeatureSpec[]) : double
-silhouettePoint(point : String[], clusterIndex : int, clusters : List<Kluster>, specs : FeatureSpec[]) : double
-intraClusterDistance(point : String[], clusterIndex : int, clusters : List<Kluster>, specs : FeatureSpec[]) : double
-minInterClusterDistance(point : String[], clusterIndex : int, clusters : List<Kluster>, specs : FeatureSpec[]) : double
+silhouettePerCluster(clusters :List<Kluster>, specs : FeatureSpec[]) : double []

FeatureSpec

+ordinalOrder: List<String>
+ordinalCardinality: Integer
+numericMin: Double
+maxSelections: Integer
+kind: VariableKind

+FeatureSpec(kind :VariableKind)
+FeatureSpec(kind : VariableKind, ordinalOrder : List<String>)
+FeatureSpec(kind : VariableKind, ordinalOrder : List<String>, ordinalCardinality : Integer, numericMin : Double, numericMax : Double, maxSelections : Integer)
+numeric(min : Double, max : Double) :FeatureSpec
+ordinal(order : java.util.List<String>) : FeatureSpec
+ordinal(order : java.util.List<String>, m : Integer) : FeatureSpec
+nominalSingle() : FeatureSpec
+nominalMulti() :FeatureSpec
+nominalMulti(maxSelections : Integer) : FeatureSpec
+freeText() : FeatureSpec

Kluster

- centroid: String[]
- members: List<String[]>

+ Kluster(centroid: String[])
+ getCentroid(): String[]
+ setCentroid(c: String[]): void
+ getMembers(): List<String[]>
+ addMember(v: String[]): void
+ clearMembers(): void
+ size() : int
+recomputeCentroid(specs : FeatureSpec[]) : boolean
-computeNumericMean(i : int) : String
-computeMode(i : int) : String
-computeMostFrequentWord(i : int) : String
+toString() : String